Identification & Ranking of India's Top 20 Startup Incubators

1. Overview

To identify and evaluate the largest startup incubator and accelerator programs in India, we conducted a multi-stage quantitative and qualitative analysis across 48 premier Indian incubators. Rather than relying on a single subjective list, we synthesized ecosystem data from government databases (DST-NIDHI, BIRAC, AIM NITI Aayog), industry reports, official incubator disclosures, and AI-assisted data aggregation.

We evaluated the ecosystem using **two distinct ranking methodologies**:

- **Methodology A: Unweighted Borda Count Index** - Balanced evaluation across Volume of startups supported till date, Capital (financial strength), & Infrastructure (Area) (1:1:1 ratio weight).
- **Methodology B: Weighted Composite Performance Index** - Capital-first approach (50/35/15 ratio).

Stage 1: Initial Discovery & Dataset Expansion (Building the Top 48 Matrix)

- **Initial Benchmarking:** We began with an initial set of 20 widely cited incubator programs (including T-Hub, NASSCOM 10,000 Startups, IIT Madras Incubation Cell, and SINE IIT Bombay).
- **Cross-Verification & Expansion:** To prevent selection bias, we cross-referenced national startup databases, academic research park registries, and AI research tools to identify additional top-tier domestic entities.
- **Universe Selection:** This expanded our analysis to **48 premier homegrown Indian incubators**, capturing institutional research parks (e.g., IITs, IIMs, CSIR-NCL), state nodal missions (e.g., KSUM, WE Hub, i-Hub), and top private VC/accelerator networks (e.g., Peak XV Surge, Venture Catalysts, 100X.VC).

Stage 2: Data Standardization Across Core Variables

To compare diverse entities (ranging from physical biotech parks to asset-light VC accelerators), we collected and standardized three comparable variables across all 48 entities:

1. **Cumulative Startups Supported / Incubated:** Total direct cohort graduations or seed-grant recipients from inception through FY 2025-26.
2. **Financial Capability & Capital Mobilized:** Total portfolio valuation, follow-on capital raised, fund AUM, or grant corpus managed.
3. **Physical Built-Up Infrastructure Area:** Total physical footprint in square feet or campus acreage

dedicated to incubation labs, maker spaces, and co-working.

Stage 3: Conversion to Borda Preference Points (1 to 48)

Because raw numbers span vastly different scales (e.g., square feet vs. rupees raised), each metric was ranked across all 48 incubators and converted into **Borda Preference Points**:

- **Rank 1 (Best in India):** 48 Points
- **Rank 48:** 1 Point

Formula: Borda Points = 49 - Rank

2. Deliverable I: Search Methodology Details & Justification

Data Source

To build our list of 48 incubators, we gathered information from three main places:

- **Government Lists:** Official records from top Indian startup programs (like DST-NIDHI, BIRAC BioNEST, and NITI Aayog's Atal Innovation Mission).
- **Official Financial Reports:** Annual reports, news releases, and public statements about how much money their startups have raised or are worth (for example, IIT Madras Research Park's ₹74,100+ Cr portfolio valuation).
- **AI & Online Directories:** Smart search tools and online startup databases that helped us double-check the numbers and sort out startups trained in quick bootcamps from those fully supported by an incubator.

Methodology Justification

No single public database tracks every type of startup support program in India. Government portals mainly keep track of academic grants (such as PRAYAS or SEED), while private venture accelerators (like Peak XV Surge or Axilor) measure success by fund size and investment deals. By combining government records, industry websites, and smart search tools, we were able to evaluate university tech parks, state-run centers, and private investors side-by-side on an equal footing.

3. Deliverable II: Dual Ranking Methodology & Justification

Methodology A: Borda Count (1:1:1 Equal Weight)

- **Formula:** Borda Point Sum = Volume Points + Financial Points + Area Points
- **Logic:** Treats physical real estate, funding deployed, and volume of founders supported as equally important (33.3% each).
- **Best Used For:** Evaluating comprehensive institutional capacity and physical infrastructure strength.

Methodology B: Weighted Composite Performance Index (50 / 35 / 15)

- **Formula:** Weighted Score = (Financial Pts × 0.50) + (Volume Pts × 0.35) + (Area Pts × 0.15)
- **Logic & Defense of Weights:**
  - **Capital Strength (50% Weight):** A startup's survival depends on funding. Measuring how much capital an incubator deploys or how much its portfolio startups are worth is the direct proof that the market values its work.
  - **Startup Volume (35% Weight):** Supporting a large number of startups proves that an incubator has reliable operating processes, a steady stream of incoming applicants, and broad network reach.
  - **Physical Footprint (15% Weight):** Most tech startups today run on software, cloud tools, and remote work, making physical office size secondary to funding and mentorship. Setting physical space at 15% gives fair credit to specialized hardware and biotech labs that need physical equipment, without unfairly penalizing modern, high-impact venture programs that operate out of smaller spaces (like Peak XV Surge or 100X.VC).

4. Deliverable III: Top 20 Ranking Comparison

Below is the comparison of **India's Top 20 Startup Incubators and Accelerators**, contrasting the results derived from **Methodology A (Unweighted Borda)** versus **Methodology B (Weighted Index)**.

Final Rank (Weighted)	Incubator / Accelerator Name	Headquarter	Startups Incubated	Volume of startups Rank	Comparable Financial Metric	Financial Rank	Physical Space (sq. ft.)	Area Rank	Method A: Unweighted Borda Rank	Method B: Final Weighted Rank Score
1	**NASSCOM 10,000 Startups**	Pan-India	11,000+	1	₹7,500+ Cr external funding raised	2	1,500,000+ sq. ft.	2	**1**	47.35
2	**IIT Madras Incubation Cell (IITMIC)**	Chennai	511+	10	₹74,100+ Cr ($8.9B) portfolio market cap	1	1,200,000 sq. ft.	6	**3**	44.10
3	**T-Hub**	Hyderabad	2,500+	5	₹3,000+ Cr facilitated capital; ₹700 Cr govt funds	3	582,689 sq. ft.	5	**2**	45.00
4	**CIIE.CO (IIMA Ventures)**	Ahmedabad	1,000+	6	₹2,500+ Cr ($300M+) capital mobilized	4	25,000 sq. ft.	24	**6**	41.30
5	**Kerala Startup Mission (KSUM)**	Kochi / TVM	6,000+	2	₹1,200+ Cr raised; ₹100+ Cr annual grant pool	12	1,000,000+ sq. ft.	4	**4**	41.70

6	**Peak XV Surge**	Bengaluru	200+	23	$1M to $3M initial check deployment	5	15,000 sq. ft.	31	**17**	33.80
7	**C-CAMP**	Bengaluru	350+	12	₹2,000+ Cr portfolio raised; ₹80 Cr direct grants	6	40,000 sq. ft.	17	**7**	39.25
8	**Venture Catalysts (VCats)**	Mumbai	200+	24	₹1,500+ Cr ($180M+) deployed capital	7	15,000 sq. ft.	32	**20**	32.30
9	**Zone Startups India**	Mumbai	550+	9	₹500+ Cr raised by alumni; ₹25L Azure credits	17	12,000 sq. ft.	32	**15**	32.55
10	**Axilor Ventures**	Bengaluru	180+	31	$100M+ Fund AUM; ₹25L seed checks	8	12,000 sq. ft.	33	**23**	29.20
11	**100Unicorns**	Mumbai	239+	19	$100M accelerator fund corpus	10	8,000 sq. ft.	37	**22**	31.80
12	**SINE (IIT Bombay)**	Mumbai	220+	21	₹1,000+ Cr ($120M+) capital raised	11	30,000 sq. ft.	23	**13**	32.70
13	**NSRCEL (IIM Bangalore)**	Bengaluru	3,000+	4	₹800+ Cr portfolio capital raised	14	30,000 sq. ft.	22	**9**	37.30
14	**i-Hub (Gujarat Student Hub)**	Ahmedabad	840+	7	₹200+ Cr state grants & PRAYAS seed funds	24	1,500,000 sq. ft.	3	**5**	34.10
15	**Venture Center (CSIR-NCL)**	Pune	750+	8	₹500+ Cr follow-on funding & BIRAC grants	16	45,000 sq. ft.	15	**8**	35.95
16	**FITT (IIT Delhi)**	New Delhi	200+	25	₹400+ Cr raised capital & IP licensing rev	18	35,000 sq. ft.	19	**19**	28.40
17	**IKP Knowledge Park**	Hyderabad	80+	37	₹500+ Cr life science grants & venture rounds	15	200-acre Park + 50K sq ft	1	**10**	28.40
18	**KIIT-TBI**	Bhubaneswar	250+	17	₹150+ Cr capital raised; ₹25+ Cr seed corpus	27	120,000 sq. ft.	9	**11**	28.20
19	**STEP - IIT Kharagpur**	Kharagpur	250+	16	₹150+ Cr capital raised; NIDHI grants	26	50,000 sq. ft.	13	**12**	28.45
20	**WE Hub**	Hyderabad	3,190+	3	₹130+ Cr direct credit & grant linkage	29	22,500 sq. ft.	26	**16**	29.55

5. Key Takeaways from the Dual Methodology

1. **Consensus Mega-Hubs: NASSCOM 10,000 Startups**, **IIT Madras Incubation Cell (IITMIC)**, and **T-Hub** rank in the Top 3 across both methodologies due to their high volume, massive financial scale, and physical presence.
2. **Impact of Methodology B (Weighted):** Private scale-up accelerators like **Peak XV Surge** (moves up from #17 to #6) and **Venture Catalysts** (moves up from #20 to #8) jump significantly under the weighted model because their high capital deployment capacity ($1M$–3M checks) is properly valued rather than penalised for smaller physical office space.

6. Annexure: Initial 48

#	Incubator / Accelerator Name	Location	1. Volume Metric			2. Financial Metric			3. Infrastructure Area			Borda Model		Weighted Model (50/35/15)	
			Cumul. Startups	Vol Rank	Vol Pts	Financial Figure	Fin Rank	Fin Pts	Area / Built-Up	Area Rank	Area Pts	Borda Sum	Borda Rank	W. Score	Final Rank
1	**NASSCOM 10,000 Startups**	Pan-India	11,000+	1	48	₹7,500+ Cr raised	2	47	1.5M+ sq. ft.	2	47	142	1	**47.35**	**1**
2	**IIT Madras Incubation Cell (IITMIC)**	Chennai	511+	10	39	₹74,100+ Cr mkt cap	1	48	1.2M sq. ft.	6	43	130	3	**44.10**	**2**
3	**T-Hub**	Hyderabad	2,500+	5	44	₹3,000+ Cr facilitated	3	46	582,689 sq. ft.	5	44	134	2	**45.00**	**3**
4	**CIIE.CO (IIMA Ventures)**	Ahmedabad	1,000+	6	43	₹2,500+ Cr mobilized	4	45	25,000 sq. ft.	24	25	113	6	**41.30**	**4**
5	**Kerala Startup Mission (KSUM)**	Kochi / TVM	6,000+	2	47	₹1,200+ Cr raised	12	37	1.0M+ sq. ft.	4	45	129	4	**41.70**	**5**
6	**Peak XV Surge**	Bengaluru	200+	23	26	$1M$–3M checks	5	44	15,000 sq. ft.	31	18	88	17	**33.80**	**6**
7	**C-CAMP**	Bengaluru	350+	12	37	₹2,000+ Cr raised	6	43	40,000 sq. ft.	17	32	112	7	**39.25**	**7**
8	**Venture Cataly**	Mumbai	200+	24	25	₹1,500+ Cr deploye	7	42	15,000 sq. ft.	32	17	84	20	**32.30**	**8**

#	Incubator / Accelerator Name	Location	1. Volume Metric			2. Financial Metric			3. Infrastructure Area			Borda Model		Weighted Model (50/35/15)	
			Cumul. Startups	Vol Rank	Vol Pts	Financial Figure	Fin Rank	Fin Pts	Area / Built-Up	Area Rank	Area Pts	Borda Sum	Borda Rank	W. Score	Final Rank
	sts (VCats)					d									
9	**Zone Startups India**	Mumbai	550+	9	40	₹500+ Cr raised	17	32	12,000 sq. ft.	32	17	89	15	**32.55**	**9**
10	**Axilor Ventures**	Bengaluru	180+	31	18	$100M + AUM	8	41	12,000 sq. ft.	33	16	75	23	**29.20**	**10**
11	**100Unicorns**	Mumbai	239+	19	30	$100M Fund	10	39	8,000 sq. ft.	37	12	81	22	**31.80**	**11**
12	**SINE (IIT Bombay)**	Mumbai	220+	21	28	₹1,000 + Cr raised	11	38	30,000 sq. ft.	23	26	92	13	**32.70**	**12**
13	**NSRCEL (IIM Bangalore)**	Bengaluru	3,000+	4	45	₹800+ Cr raised	14	35	30,000 sq. ft.	22	27	107	9	**37.30**	**13**
14	**i-Hub (Gujarat Student Hub)**	Ahmedabad	840+	7	42	₹200+ Cr grants	24	25	1.5M sq. ft.	3	46	113	5	**34.10**	**14**
15	**Venture Center (CSIR-NCL)**	Pune	750+	8	41	₹500+ Cr raised	16	33	45,000 sq. ft.	15	34	108	8	**35.95**	**15**
16	**FITT (IIT Delhi)**	New Delhi	200+	25	24	₹400+ Cr raised	18	31	35,000 sq. ft.	19	30	85	19	**28.40**	**16**
17	**IKP Knowledge Park**	Hyderabad	80+	37	12	₹500+ Cr grants	15	34	200-acre Park	1	48	94	10	**28.40**	**17**
18	**KIIT-TBI**	Bhubaneswar	250+	17	32	₹150+ Cr raised	27	22	120,000 sq. ft.	9	40	94	11	**28.20**	**18**
19	**STEP - IIT Kharagpur**	Kharagpur	250+	16	33	₹150+ Cr raised	26	23	50,000 sq. ft.	13	36	92	12	**28.45**	**19**
20	**WE Hub**	Hyderabad	3,190+	3	46	₹130+ Cr credit	29	20	22,500 sq. ft.	26	23	89	16	**29.55**	**20**
21	**SIIC (IIT Kanpur)**	Kanpur	110+	34	15	₹300+ Cr raised	22	27	150,000 sq. ft.	8	41	83	21	**24.90**	**21**
22	**iCreate**	Ahmedabad	220+	22	27	₹50L seed+₹10Cr	35	14	210,000 sq. ft.	7	42	83	21	**22.75**	**22**

#	Incubator / Accelerator Name	Location	1. Volume Metric			2. Financial Metric			3. Infrastructure Area			Borda Model		Weighted Model (50/35/15)	
			Cumul. Startups	Vol Rank	Vol Pts	Financial Figure	Fin Rank	Fin Pts	Area / Built-Up	Area Rank	Area Pts	Borda Sum	Borda Rank	W. Score	Final Rank
23	**RiiDL (Somaiya)**	Mumbai	300+	15	34	₹100+ Cr raised	28	21	15,000 sq. ft.	29	20	75	23	**25.40**	**23**
24	**Social Alpha (FISE)**	Bengaluru	200+	26	23	₹300+ Cr pool	21	28	20,000 sq. ft.	28	21	72	24	**25.20**	**24**
25	**BITS Pilani TBI (PIEDS)**	Pilani	200+	28	21	₹100+ Cr raised	31	18	45,000 sq. ft.	16	33	72	24	**21.30**	**25**
26	**India Accelerator**	Gurugram	100+	35	14	₹200+ Cr AUM	25	24	25,000 sq. ft.	25	24	62	27	**20.50**	**26**
27	**Amity Innovation Incubator**	Noida	300+	13	36	₹50+ Cr raised	36	13	20,000 sq. ft.	27	22	71	25	**22.40**	**27**
28	**Villgro Innovations**	Chennai	300+	14	35	₹75+ Cr grant	32	17	~6,000 sq. ft.	40	9	61	28	**22.10**	**28**
29	**100X.VC**	Mumbai	200+	27	22	₹1.25 Cr checks	30	19	8,000 sq. ft.	38	11	52	34	**18.85**	**29**
30	**GSF Accelerator**	New Delhi	230+	20	29	$50M+ raised	34	15	10,000 sq. ft.	34	15	59	30	**19.90**	**30**
31	**Deshpande Startups**	Hubballi	200+	29	20	₹10L–₹50L seeds	37	12	100,000 sq. ft.	10	39	71	25	**18.85**	**31**
32	**GoMassive Earth Network**	Gurugram	400+	11	38	$10M+ climate	38	11	Virtual Model	43	6	55	31	**19.70**	**32**
33	**JSS ATE STEP**	Noida	180+	33	16	₹50+ Cr raised	36	13	40,000 sq. ft.	18	31	60	29	**16.75**	**33**
34	**Advant Edge Founders**	Noida	53+	41	8	$30M+ Fund AUM	23	26	~4,000 sq. ft.	42	7	41	36	**16.85**	**34**
35	**JioGen Next**	Navi Mumbai	247+	18	31	Reliance Jio	44	5	15,000 sq. ft.	30	19	55	31	**16.20**	**35**
36	**Massive Earth Foundation**	Gurugram	180+	32	17	$15M+ climate	33	16	Virtual Model	44	5	38	37	**14.70**	**36**
37	**Amrita**	Kollam	200+	30	19	₹20L–₹	39	10	25,000	25	24	53	33	**15.25**	**37**

#	Incubator / Accelerator Name	Location	1. Volume Metric			2. Financial Metric			3. Infrastructure Area			Borda Model		Weighted Model (50/35/15)	
			Cumul. Startups	Vol Rank	Vol Pts	Financial Figure	Fin Rank	Fin Pts	Area / Built-Up	Area Rank	Area Pts	Borda Sum	Borda Rank	W. Score	Final Rank
	TBI					50L seed			sq. ft.						
38	**Faad Network**	Gurugram	96+	36	13	₹75 Cr+ deployed	31	18	~3,000 sq. ft.	45	4	35	38	**14.15**	**38**
39	**AIC RAISE**	Coimbatore	80+	38	11	₹10 Cr grant	41	8	10,000 sq. ft.	35	14	33	40	**9.95**	**39**
40	**Incubate Fund India**	Bengaluru	40+	42	7	$50M+ VC Fund	35	14	~3,000 sq. ft.	46	3	24	43	**9.90**	**40**
41	**CIE (IIIT Hyderabad)**	Hyderabad	35+	45	4	₹100+ Cr raised	40	9	20,000 sq. ft.	28	21	34	39	**9.05**	**41**
42	**RKVY-RAFTAAR (IIT BHU)**	Varanasi	59+	41	8	₹15+ Cr grants	42	7	10,000 sq. ft.	36	13	28	41	**8.25**	**42**
43	**TLabs (Times Internet)**	Pan-India	63+	39	10	Corporate backed	43	6	8,000 sq. ft.	39	10	26	42	**8.00**	**43**
44	**IIT Mandi Catalyst**	Mandi	35+	44	5	₹10+ Cr seed	45	4	10,000 sq. ft.	36	13	22	44	**5.70**	**44**
45	**KIT's Innovation Foundation**	Kolhapur	40+	43	6	₹5 Cr+ grants	46	3	8,000 sq. ft.	41	8	17	46	**4.80**	**45**
46	**TIDES (IIT Roorkee)**	Roorkee	30+	46	3	₹20+ Cr grants	47	2	12,000 sq. ft.	33	16	21	45	**4.45**	**46**
47	**IIM Calcutta Innovation Park**	Kolkata	25+	47	2	₹150+ Cr capital	48	1	15,000 sq. ft.	31	18	21	45	**3.90**	**47**
48	**The Entrepreneur Zone (TEZ)**	Hyderabad	15+	48	1	Bootcamps	48	1	Virtual Model	41	8	10	47	**2.05**	**48**

7. Annexure: Initial Automated Data Scraping & Pipeline Limitations

---

Initial Automated Data Extraction Trial

Prior to finalizing the current methodology, an automated web scraping pipeline was constructed to harvest metrics directly from the websites of an initial list of **313 incubators** compiled from public sources (including AIM, MeitY, and DPIIT directories).

LINK:
https://docs.google.com/spreadsheets/d/1LrVuC4aOhG2kiGtBM5Sr5TxwM_XB--byeZ40aD08XCw/edit?usp=sharing

The Python script below was deployed to systematically crawl homepage and inner impact/portfolio pages using regular expressions, keyword prioritizations, and cached HTML parsing:

```
import re                    # "regular expressions": pattern matching in text
import os                    # file and folder handling
import time                  # for polite delays between requests
import hashlib               # to make safe file names for the saved pages
import logging               # prints progress messages
from urllib.parse import urljoin, urlparse   # tools for working with web addresses
from urllib.robotparser import RobotFileParser  # reads each site's robots.txt rules

import requests              # downloads web pages
import pandas as pd          # builds tables and writes Excel
from bs4 import BeautifulSoup  # reads the structure of an HTML page

# ---------------------------------------------------------------------------
# 1. SETTINGS
# ---------------------------------------------------------------------------
INPUT_FILE = "incubators.txt"      # pasted list
OUTPUT_DIR = "output"              # results directory
CACHE_DIR = "html_cache"           # downloaded pages cache

START = 0          # first row to process
END = 10           # last row
MAX_PAGES_PER_SITE = 3   # homepage + up to 5 inner pages per incubator
DELAY_SECONDS = 1.5      # pause between requests
TIMEOUT_SECONDS = 20     # request timeout
RESPECT_ROBOTS_TXT = True  # skip disallowed pages

USER_AGENT = ("Mozilla/5.0 (compatible; IIMB-RA-research-bot; "
              "contact: ge24arushi@mse.ac.in)")
```

```
LINK_KEYWORDS = [
    "impact", "portfolio", "startups", "start-ups", "our-startups", "incubatees",
    "success", "stories", "alumni", "about", "overview", "who-we-are",
    "facilit", "infrastructure", "lab", "funding", "fund", "grant",
    "program", "achievement", "statistic", "numbers", "highlights", "annual",
]

SKIP_EXTENSIONS = (".jpg", ".jpeg", ".png", ".gif", ".svg", ".webp", ".zip",
                   ".doc", ".docx", ".xls", ".xlsx", ".ppt", ".pptx", ".mp4", ".mp3")

logging.basicConfig(level=logging.INFO, format="%(asctime)s  %(message)s",
                    datefmt="%H:%M:%S")
log = logging.getLogger("finder")

# ---------------------------------------------------------------------------
# 2. REGEX PATTERNS & METRIC LOGIC
# ---------------------------------------------------------------------------
NUM = r"(?P<num>\d{1,3}(?:,\d{2,3})+|\d+(?:\.\d+)?)"
SUFFIX = r"(?P<plus>\s*\+)?(?P<k>\s*[kK]\b)?"
FILLER = r"(?:\s+[A-Za-z\-&/]+){0,3}?\s+"
FILLER_AFTER = (r"(?:\s+(?:incubated|supported|funded|created|raised|total|nurtured|"
                r"mentored|hosted|enabled|trained|so|far|till|date|over|more|than|"
                r"nearly|about|approx|approximately|currently|active)){0,4}")

COUNT_CATEGORIES = {
    "startups":        r"(?:start-?ups?|incubatees?|portfolio\s+companies|ventures)",
    "jobs":            r"(?:jobs|employment|livelihoods?)",
    "patents":         r"(?:patents?|IPRs?)",
    "labs":            r"(?:labs?|laborator(?:y|ies)|equipment)",
    "seats":           r"(?:seats|seating|workstations?|desks)",
    "mentors":         r"(?:mentors|advisors|domain\s+experts)",
    "investors":       r"(?:investors|VCs|angels)",
    "cohorts":         r"(?:cohorts|batches)",
    "unicorns_exits":  r"(?:unicorns?|exits?|acquisitions?|IPOs?)",
    "alumni":          r"(?:alumni|graduat(?:es|ed))",
    "partners":        r"(?:partners|corporates)",
}

MONEY_UNITS = r"(?P<unit>crores?|cr|lakhs?|lacs?|million|mn|billion|bn|thousand|k|m|b)"
MONEY_WITH_SYMBOL = (r"(?P<cur>₹|Rs\.?|INR|USD|US\$|\$)\s*" + NUM +
                     r"\s*" + MONEY_UNITS + r"?\b")
MONEY_NO_SYMBOL = NUM + r"\s*" + r"(?P<unit>crores?|cr|lakhs?|lacs?|million|mn|billion|bn)\b"
FUNDING_WORDS = (r"(?:fund|funding|funds|grant|grants|raised|corpus|invest|investment|"
                 r"invested|disburs|sanction|capital|seed|mobili[sz]ed)")
RAISED_WORDS = r"(?:raised|follow-?on|mobili[sz]ed|attracted|secured)"
HELD_WORDS = r"(?:grant|corpus|fund\s+size|seed\s+fund|disburs|sanction|supported\s+with|worth)"

AREA_PATTERN = (NUM + r"\s*(?P<unit>sq\.?\s*ft\.?|sq\.?\s*feet|square\s+feet|square\s+foot|"
                r"sq\.?\s*m(?:eters?|etres?)?|square\s+m(?:eters?|etres?)|acres?)")

FLAGS = re.IGNORECASE
COUNT_REGEXES = {}
for _name, _kw in COUNT_CATEGORIES.items():
```

```
    before = re.compile(NUM + SUFFIX + FILLER + r"(?P<kw>" + _kw + r")\b", FLAGS)
    after = re.compile(r"(?P<kw>" + _kw + r")\b" + FILLER_AFTER + r"\s*[:\-\u2013]?\s*"
                       + NUM + SUFFIX, FLAGS)
    COUNT_REGEXES[_name] = (before, after)

MONEY_REGEXES = [re.compile(MONEY_WITH_SYMBOL, FLAGS), re.compile(MONEY_NO_SYMBOL, FLAGS)]
AREA_REGEX = re.compile(AREA_PATTERN, FLAGS)
FUNDING_WORD_REGEX = re.compile(FUNDING_WORDS, FLAGS)
RAISED_REGEX = re.compile(RAISED_WORDS, FLAGS)
HELD_REGEX = re.compile(HELD_WORDS, FLAGS)

def to_number(text, k_flag=False):
    try:
        value = float(text.replace(",", ""))
    except ValueError:
        return None
    if k_flag:
        value *= 1000
    return value

MONEY_MULTIPLIER = {
    "crore": 1e7, "crores": 1e7, "cr": 1e7,
    "lakh": 1e5, "lakhs": 1e5, "lac": 1e5, "lacs": 1e5,
    "million": 1e6, "mn": 1e6, "m": 1e6,
    "billion": 1e9, "bn": 1e9, "b": 1e9,
    "thousand": 1e3, "k": 1e3,
}

def snippet(text, start, end, width=110):
    s = max(0, start - width)
    e = min(len(text), end + width)
    return text[s:e].replace("\n", " ").strip()

def same_sentence(text, start, end):
    s0 = text.rfind(". ", 0, start)
    s0 = 0 if s0 == -1 else s0 + 2
    e0 = text.find(". ", end)
    e0 = len(text) if e0 == -1 else e0
    return text[s0:e0]

def looks_like_year(value, has_plus, has_comma):
    return (value is not None and 1990 <= value <= 2035
            and not has_plus and not has_comma)

def extract_hits(text, page_url):
    hits = []
    # (a) counts
    for category, (before, after) in COUNT_REGEXES.items():
        for regex in (before, after):
            for m in regex.finditer(text):
                raw = m.group("num")
                value = to_number(raw, bool(m.group("k")))
                if value is None or value <= 0:
                    continue
```

```
            if looks_like_year(value, bool(m.group("plus")), "," in raw):
                continue
            before_num = text[max(0, m.start("num") - 6): m.start("num")]
            if re.search(r"(₹|rs\.?|inr|usd|\$)\s*$", before_num, re.IGNORECASE):
                continue
            hits.append({
                "category": category, "value": value, "unit_or_currency": "count",
                "matched_text": m.group(0).strip(), "context": snippet(text, m.start(), m.end()),
                "page_url": page_url,
            })
    # (b) money
    money_spans = []
    for regex in MONEY_REGEXES:
        for m in regex.finditer(text):
            if any(m.start() < e and m.end() > s0 for s0, e in money_spans):
                continue
            money_spans.append((m.start(), m.end()))
            context = snippet(text, m.start(), m.end(), width=90)
            near = same_sentence(text, m.start(), m.end())
            if not FUNDING_WORD_REGEX.search(context):
                continue
            raw = m.group("num")
            unit = (m.groupdict().get("unit") or "").lower()
            value = to_number(raw)
            if value is None:
                continue
            value *= MONEY_MULTIPLIER.get(unit, 1)
            currency = (m.groupdict().get("cur") or "INR(assumed)").replace("Rs.", "INR")
            currency = "INR" if currency in ("₹", "Rs") else currency
            if RAISED_REGEX.search(near):
                category = "funding_raised_by_startups"
            elif HELD_REGEX.search(near):
                category = "funds_held_or_granted"
            else:
                category = "funding_other"
            hits.append({
                "category": category, "value": value, "unit_or_currency": currency,
                "matched_text": m.group(0).strip(), "context": context, "page_url": page_url,
            })
    # (c) area
    for m in AREA_REGEX.finditer(text):
        value = to_number(m.group("num"))
        if value is None:
            continue
        hits.append({
            "category": "floor_area", "value": value, "unit_or_currency": re.sub(r"\s+", " ", m.group("unit").lower()),
            "matched_text": m.group(0).strip(), "context": snippet(text, m.start(), m.end()), "page_url": page_url,
        })
    return hits

# ---------------------------------------------------------------------------
# 3. INPUT, FETCH, AND EXECUTION PIPELINE
# ---------------------------------------------------------------------------
def read_input(path):
```

```
    rows, seen = [], {}
    with open(path, encoding="utf-8") as f:
        for line in f:
            m = re.search(r"https?://\S+", line)
            if not m:
                continue
            url = m.group(0).strip()
            before = line[:m.start()].strip()
            parts = re.split(r"\s+", before, maxsplit=1)
            if len(parts) == 2 and parts[0].isdigit():
                sno, name = int(parts[0]), parts[1].strip()
            else:
                sno, name = None, before
            if url in seen:
                seen[url]["also_listed_as"] += " | " + name
                continue
            row = {"s_no": sno, "name": name, "website": url, "also_listed_as": ""}
            seen[url] = row
            rows.append(row)
    return rows

session = requests.Session()
session.headers.update({"User-Agent": USER_AGENT})
_robots_cache = {}
_last_request_time = {}

def domain_of(url):
    return urlparse(url).netloc.lower().replace("www.", "")

def allowed_by_robots(url):
    if not RESPECT_ROBOTS_TXT:
        return True
    parsed = urlparse(url)
    base = f"{parsed.scheme}://{parsed.netloc}"
    if base not in _robots_cache:
        rp = RobotFileParser()
        try:
            r = session.get(base + "/robots.txt", timeout=TIMEOUT_SECONDS)
            if r.status_code == 200:
                rp.parse(r.text.splitlines())
            else:
                rp.parse([])
        except requests.RequestException:
            rp.parse([])
        _robots_cache[base] = rp
    return _robots_cache[base].can_fetch(USER_AGENT, url)

def polite_wait(domain):
    last = _last_request_time.get(domain, 0)
    wait = DELAY_SECONDS - (time.time() - last)
    if wait > 0:
        time.sleep(wait)
    _last_request_time[domain] = time.time()
```

```
def cache_path(url):
    return os.path.join(CACHE_DIR, hashlib.sha1(url.encode()).hexdigest() + ".html")

def fetch(url):
    path = cache_path(url)
    if os.path.exists(path):
        with open(path, encoding="utf-8", errors="ignore") as f:
            return f.read(), "cached"
    if not allowed_by_robots(url):
        return None, "blocked_by_robots_txt"
    domain = domain_of(url)
    note = ""
    for attempt in range(3):
        try:
            polite_wait(domain)
            try:
                r = session.get(url, timeout=TIMEOUT_SECONDS)
            except requests.exceptions.SSLError:
                r = session.get(url, timeout=TIMEOUT_SECONDS, verify=False)
                note = "ssl_certificate_problem_ignored"
            if r.status_code == 200:
                if not r.encoding or r.encoding.lower() == "iso-8859-1":
                    r.encoding = r.apparent_encoding
                with open(path, "w", encoding="utf-8") as f:
                    f.write(r.text)
                return r.text, note or "ok"
            note = f"http_{r.status_code}"
            if r.status_code in (403, 404, 410):
                break
        except requests.RequestException as e:
            note = f"error_{type(e).__name__}"
        time.sleep(2 * (attempt + 1))
    return None, note

def html_to_text(html):
    soup = BeautifulSoup(html, "html.parser")
    for tag in soup(["script", "style", "noscript", "svg"]):
        tag.decompose()
    for tag in soup.find_all(True):
        for attr in ("data-count", "data-to", "data-target", "data-number",
                     "data-value", "data-end", "data-counter"):
            val = tag.get(attr)
            if val and re.fullmatch(r"\d[\d,\.]*", str(val)):
                if tag.get_text(strip=True) in ("", "0", "00"):
                    tag.string = str(val)
    text = soup.get_text(" ")
    return re.sub(r"\s+", " ", text)

def find_links(html, base_url):
    soup = BeautifulSoup(html, "html.parser")
    base_domain = domain_of(base_url)
    scored, pdfs, seen = [], [], set()
    for a in soup.find_all("a", href=True):
        href = a["href"].strip()
```

```
        if href.startswith(("mailto:", "tel:", "javascript:", "#")):
            continue
        full = urljoin(base_url, href).split("#")[0]
        label = (a.get_text(" ", strip=True) + " " + full).lower()
        if full.lower().endswith(".pdf"):
            if re.search(r"report|impact|annual|brochure|portfolio", label):
                pdfs.append((a.get_text(" ", strip=True)[:80], full))
            continue
        if full.lower().endswith(SKIP_EXTENSIONS):
            continue
        if domain_of(full) != base_domain or full in seen:
            continue
        seen.add(full)
        for rank, kw in enumerate(LINK_KEYWORDS):
            if kw in label:
                scored.append((rank, full))
                break
    scored.sort(key=lambda x: x[0])
    ordered = []
    for _, link in scored:
        if link.rstrip("/") != base_url.rstrip("/") and link not in ordered:
            ordered.append(link)
    return ordered, pdfs

def process_incubator(row):
    url = row["website"]
    all_hits, pdf_rows, pages_ok, notes = [], [], 0, []
    html, note = fetch(url)
    notes.append(f"home:{note}")
    if html is None:
        return all_hits, pdf_rows, {"pages_fetched": 0, "status": note, "possibly_javascript_site": ""}
    home_text = html_to_text(html)
    js_suspect = len(home_text) < 600
    pages_ok += 1
    all_hits += [dict(h, page_type="home") for h in extract_hits(home_text, url)]
    inner_links, pdfs = find_links(html, url)
    pdf_rows += [{"s_no": row["s_no"], "name": row["name"], "pdf_label": l, "pdf_url": u}
                 for l, u in pdfs[:10]]
    for link in inner_links[: MAX_PAGES_PER_SITE - 1]:
        page_html, page_note = fetch(link)
        notes.append(f"{link}:{page_note}")
        if page_html is None:
            continue
        pages_ok += 1
        text = html_to_text(page_html)
        all_hits += [dict(h, page_type="inner") for h in extract_hits(text, link)]
    status = "ok" if pages_ok else "failed"
    return all_hits, pdf_rows, {"pages_fetched": pages_ok, "status": status,
                                "possibly_javascript_site": "YES" if js_suspect else "",
                                "fetch_notes": " ; ".join(notes)[:500]}

SUMMARY_CATEGORIES = ["startups", "funding_raised_by_startups", "funds_held_or_granted",
                      "jobs", "patents", "labs", "seats", "floor_area", "mentors",
                      "investors", "cohorts", "unicorns_exits", "alumni", "partners"]
```

```
def build_summary(rows, hits_df, log_rows):
    summary = []
    for row in rows:
        rec = {"s_no": row["s_no"], "name": row["name"], "website": row["website"], "also_listed_as": row["also_listed_as"]}
        rec.update(log_rows.get(row["website"], {}))
        sub = hits_df[hits_df["website"] == row["website"]] if len(hits_df) else hits_df
        for cat in SUMMARY_CATEGORIES:
            cat_hits = sub[sub["category"] == cat] if len(sub) else sub
            if len(cat_hits):
                best = cat_hits.sort_values("value", ascending=False).iloc[0]
                rec[f"{cat}_max"] = best["value"]
                rec[f"{cat}_unit"] = best["unit_or_currency"]
                rec[f"{cat}_n_matches"] = len(cat_hits)
                rec[f"{cat}_snippet"] = best["context"]
            else:
                rec[f"{cat}_max"] = None
                rec[f"{cat}_n_matches"] = 0
        summary.append(rec)
    return pd.DataFrame(summary)

def save_outputs(rows, hits, pdf_rows, log_rows):
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    hits_df = pd.DataFrame(hits)
    if len(hits_df):
        hits_df = hits_df.drop_duplicates(subset=["website", "category", "value", "unit_or_currency"])
    summary_df = build_summary(rows, hits_df, log_rows)
    log_df = pd.DataFrame([dict(website=k, **v) for k, v in log_rows.items()])
    pdf_df = pd.DataFrame(pdf_rows)

    with pd.ExcelWriter(os.path.join(OUTPUT_DIR, "incubator_metrics.xlsx")) as writer:
        summary_df.to_excel(writer, sheet_name="summary", index=False)
        hits_df.to_excel(writer, sheet_name="all_hits", index=False)
        pdf_df.to_excel(writer, sheet_name="report_pdf_links", index=False)
        log_df.to_excel(writer, sheet_name="fetch_log", index=False)
    summary_df.to_csv(os.path.join(OUTPUT_DIR, "summary.csv"), index=False)
    hits_df.to_csv(os.path.join(OUTPUT_DIR, "all_hits.csv"), index=False)

def main():
    os.makedirs(CACHE_DIR, exist_ok=True)
    rows = read_input(INPUT_FILE)[START:END]
    log.info("Loaded %d unique incubator websites", len(rows))
    all_hits, all_pdfs, log_rows = [], [], {}
    for i, row in enumerate(rows, start=1):
        log.info("[%d/%d] %s", i, len(rows), row["name"][:60])
        try:
            hits, pdfs, status = process_incubator(row)
        except Exception as e:
            hits, pdfs, status = [], [], {"pages_fetched": 0, "status": f"crash_{type(e).__name__}"}
        for h in hits:
            h.update({"website": row["website"], "s_no": row["s_no"], "name": row["name"]})
            all_hits += hits
            all_pdfs += pdfs
            log_rows[row["website"]] = status
```

```
            if i % 20 == 0:
                save_outputs(rows, all_hits, all_pdfs, log_rows)
    save_outputs(rows, all_hits, all_pdfs, log_rows)

if __name__ == "__main__":
    requests.packages.urllib3.disable_warnings()
    main()
```

Why Automated Extraction Proved Inadequate

While the automated script quickly scanned hundreds of websites, the extracted numbers were inconsistent and unreliable due to three main web design issues:

- **Hidden Website Data:** Many modern incubator websites load their numbers dynamically using interactive scripts or frameworks (like React or Vue). Static page downloaders miss this content entirely, recording zeros instead of actual metrics.
- **Inconsistent Definitions:** Incubator websites report metrics differently. Some group short-term bootcamp attendees together with fully incubated startups, while others mix annual seed grants with total portfolio valuations.
- **Outdated & Broken Pages:** Many university and government-backed websites had broken security certificates, inactive links, or metric pages that had not been updated in years.

Because of these gaps, we replaced automated scraping with a manual, multi-source verification process. This ensured every figure across the 48 entities was accurate, up-to-date, and standardized.